VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

School of Computer Science and Engineering

DATABASE MANAGEMENT SYSTEMS

Mini-Project Report

ENHANCED CANDIDATE RECRUITMENT AND AUTOMATED SYSTEM (ECRAS)

ER/EER Modelling and Relational Normalization (UNF to BCNF)

By -

Ishan Vaidya (25BCE1197)

M Yuvaraj (25BCE5317)

Hemchand Mouli (25BCE1258)

Under the guidance of

Dr. Sharmila Devi S

Assistant Professor Senior Grade 1, SCOPE

Academic Year 2026 – 2027

CERTIFICATE

This is to certify that the report titled "Enhanced Candidate Recruitment and Automated System (ECRAS)" is a bona fide record of the DBMS mini-project carried out by Ishan Vaidya (25BCE1197), M Yuvaraj (25BCE5317) and Hemchand Mouli (25BCE1258), under my supervision, at Vellore Institute of Technology, Chennai. The work has not been submitted elsewhere for the award of any degree or diploma.

Guide Signature: _____ Date: _____

DECLARATION

We declare that this project report is our own work, carried out under the guidance of Dr. Sharmila Devi S, and that all sources referred to have been duly acknowledged.

Student Signatures:

1. _____
2. _____
3. _____

ACKNOWLEDGEMENT

We thank our guide, Dr. Sharmila Devi S, for her guidance and support through this project, and the School of Computer Science and Engineering, VIT Chennai, for the resources made available to us.

ABSTRACT

Most Applicant Tracking Systems still lean on plain keyword matching, so a resume that says “led a cross-functional team” instead of “managed a group” can get filtered out even though the candidate is qualified. This project, ECRAS, tries to fix that by combining lexical search (BM25) with semantic vector search, fusing the two rankings together, and adding an automated interview stage that transcribes and scores answers against a fixed rubric. A bias-auditing layer sits underneath all of this, tracking selection and impact ratios per demographic group in line with NYC Local Law 144 and the EU AI Act.

The larger part of this report is a database design exercise. We draw the ER/EER model for ECRAS, map it to a relational schema, and then work through the normal forms one at a time, starting from an unnormalized flat report and going all the way up to BCNF, so that every relation in the final schema can be shown to be free of redundancy and update anomalies.

TABLE OF CONTENTS

1. Introduction 5
2. Literature Survey 6
3. System Architecture 7
4. Requirement Analysis 8
5. Module Description 9
6. Database Design and Normalization 10
6.1 ER Model 10
6.2 Relational Schema 11
6.3 Functional Dependencies and Normalization (UNF to BCNF) 11
6.4 Constraints and Referential Integrity 19
7. Technology Stack 20
8. Implementation (SQL, PL/SQL and Application) 21
8.1 Implementation Overview and Architecture 21
8.2 Database Implementation (SQL) 22
8.3 PL/SQL Implementation 24
8.4 Backend (API) Integration 26
8.5 Frontend and User Interface 26
8.6 UI/UX and Engineering Standards 33
8.7 Testing and Verification 34
8.8 Traceability to Design Requirements 35
8.9 Limitations and Assumptions 36
9. Conclusion and Future Scope 37
References 37

LIST OF FIGURES

Fig. 8.1 Three-tier architecture of the ECRAS implementation 21
Fig. 8.2 Job Postings dashboard 27
Fig. 8.3 Create Job Posting form 28
Fig. 8.4 Candidates dashboard 28
Fig. 8.5 Add Candidate form 29
Fig. 8.6 Application Ranking dashboard (job selection) 30
Fig. 8.7 Ranked applicants for the Truck Driver Job 30
Fig. 8.8 Interview Transcripts dashboard (session lookup) 31
Fig. 8.9 Transcript turns and session summary for Session ID 2 31
Fig. 8.10 Compliance and Bias Audit dashboard (job selection) 32
Fig. 8.11 Computed bias audit for the Truck Driver Job (ID 1001) 33

1. INTRODUCTION

1.1 Background

Hiring today runs almost entirely through software. An ATS decides which resumes a recruiter even gets to see, and for a long time that decision was made by a simple inverted-index keyword search. That approach is fast but linguistically dumb: it cannot tell that “led a cross-functional team” and “managed a multidisciplinary group” mean roughly the same thing. Two things have changed since then. Transformer embeddings now make semantic search practical at scale, and regulators such as New York City (Local Law 144) and the EU (the AI Act) now treat hiring algorithms as high-risk systems that must be audited for bias. ECRAS was designed around both of these from the start.

1.2 Problems with Existing Systems

- Keyword-only filters reject qualified candidates who phrase things differently from the job description.
- Manual screening at scale pushes recruiters toward shortcuts that do not track well with actual job performance.
- Interviews run without a fixed rubric are hard to compare or defend later.
- Most legacy ATS platforms have no built-in bias auditing, which is now a legal exposure and not just a nice-to-have.

1.3 Objectives and Scope

ECRAS covers resume ingestion, hybrid ranking, an AI-assisted interview, and a compliance layer that computes selection and impact ratios on an ongoing basis. The final hiring decision always stays with a human recruiter; payroll and onboarding are out of scope. The goals set for the system were to cut down false-negative screening, standardise interview scoring, keep disparate-impact monitoring running at all times, and stay under a 300 ms retrieval latency budget.

2. LITERATURE SURVEY

Early ATS platforms used relational full-text search, things like PostgreSQL tsvector or Lucene/Solr with BM25 scoring. These are still common because they are fast and easy to explain, but they miss synonyms and paraphrasing. Sentence-BERT (Reimers and Gurevych, 2019) changed this by producing embeddings where cosine similarity tracks semantic similarity, which is what makes dense retrieval practical. ColBERT's late-interaction scoring (Khattab and Zaharia, 2020) goes a step further by keeping a separate embedding per token instead of collapsing a document into one vector. This costs more compute, but that is usually acceptable if it only runs on a short reranking list.

On the regulation side, NYC Local Law 144 (2023) requires an independent bias audit for any automated hiring tool, including publishing selection and scoring rates by demographic group. The EU AI Act classifies recruitment AI as high-risk under Annex III, which brings documentation and human-oversight obligations along with it. ECRAS treats both of these as requirements from day one instead of something added later.

Comparative snapshot

Feature	**Traditional ATS**	**Modern AI Vendor**	**ECRAS**
Matching	Boolean / TF-IDF	Single dense embedding	BM25 + dense + RRF + rerank
Interview	Manual scheduling only	Recorded video, manual review	Live WebRTC + Whisper + rubric scoring
Bias audit	None	Ad hoc / vendor-internal	Continuous SR / ScR / IR
Explainability	N/A	Opaque single score	Evidence-linked per criterion

3. SYSTEM ARCHITECTURE

ECRAS is split into five cooperating parts: resume parsing, hybrid retrieval and reranking, the interview service, a scoring engine, and the compliance/governance engine. A FastAPI layer coordinates all of this, backed by PostgreSQL for relational state, Redis for session and cache data, and Qdrant as the vector store. Every service call is written to an append-only audit table that the governance engine reads from.

3.1 Retrieval Pipeline

A query hits a BM25 sparse index and a Qdrant dense index at the same time. Reciprocal Rank Fusion merges the two ranked lists, and the top candidates then pass through a cross-encoder reranker for a final precision pass. Keeping the expensive reranking step limited to a short list is what keeps first-stage latency low.

3.2 Interview and Scoring

Four small agents run the interview. One picks the next rubric-aligned question, one screens for prompt injection or attempts to draw out protected-characteristic information, one scores the answer with a BERT-large rubric model, and one compiles the final report. Every score shown to a recruiter is tied to the exact transcript span that produced it, which is what the Annex III human-oversight requirement actually asks for.

4. REQUIREMENT ANALYSIS

4.1 Functional Requirements

ID	Requirement
FR-1	Ingest resumes (PDF/DOCX) and split them into structured chunks.
FR-2	Generate embeddings per chunk and index them with HNSW.
FR-3	Run hybrid BM25 and dense retrieval, fused with RRF.
FR-4	Rerank the fused shortlist with a cross-encoder model.
FR-5	Run a live WebRTC interview with real-time transcription.
FR-6	Score every answer against a fixed rubric.
FR-7	Produce an evidence-linked scorecard per candidate.
FR-8	Compute Selection Rate, Scoring Rate and Impact Ratio per group.
FR-9	Export a bias-audit report for Local Law 144 disclosure.
FR-10	Log every automated decision and override for traceability.

4.2 Non-Functional Requirements

- Performance: sub-300 ms P95 retrieval latency for up to 100,000 resumes.
- Scalability: FastAPI and vector-index tiers scale independently behind a load balancer.
- Security: OAuth2/OIDC with role-based access for recruiter, candidate and auditor roles.
- Privacy: GDPR/CCPA-aligned access, rectification and erasure workflows.

5. MODULE DESCRIPTION

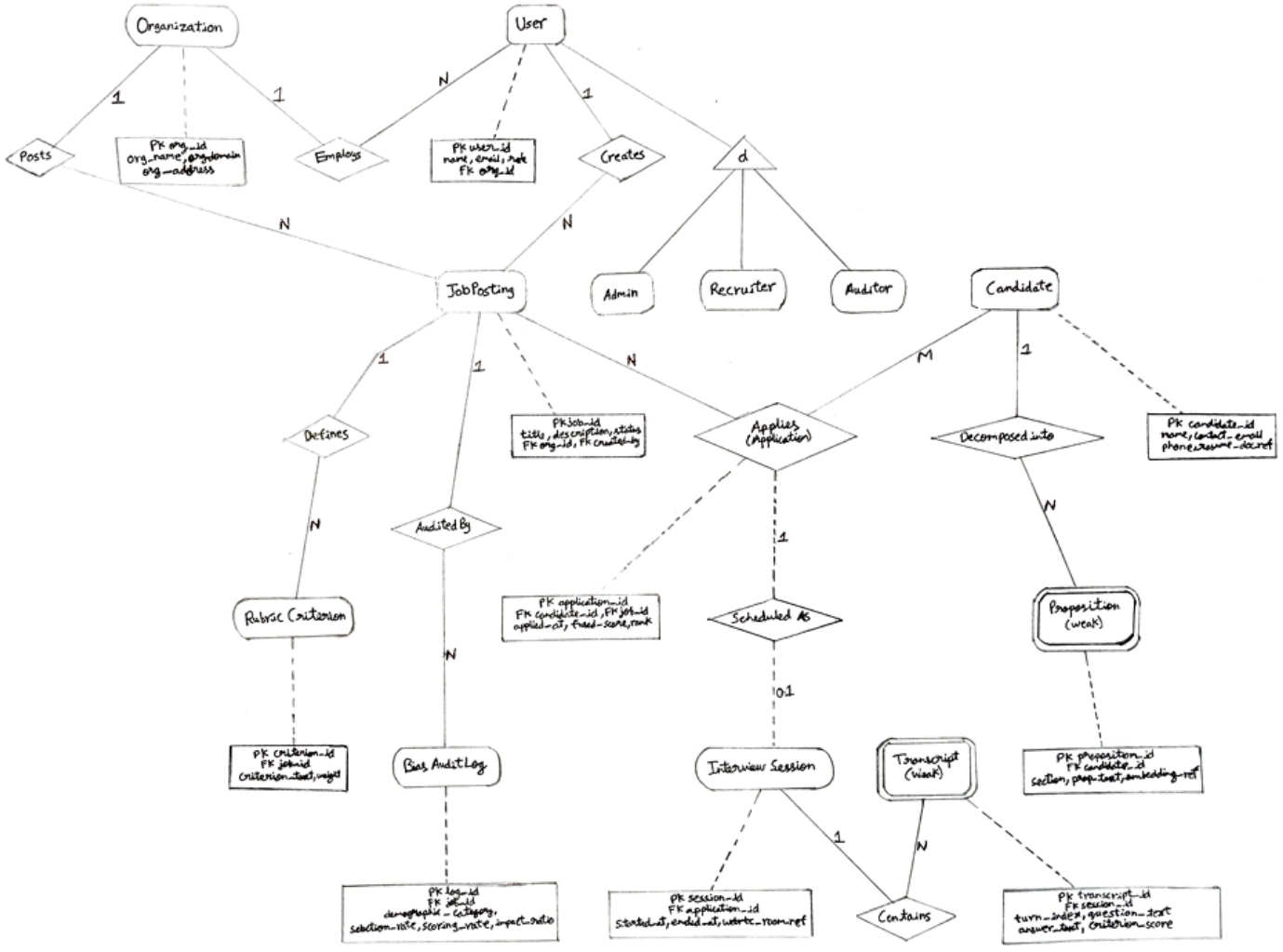
Module	What it does	One limitation
Resume Parser	Extracts text, tags sections, splits bullets into atomic chunks so embeddings do not get diluted.	Struggles with scanned or poorly formatted resumes.
Hybrid Search Engine	Runs BM25 and dense retrieval in parallel, fuses via RRF, reranks the top-K.	RRF can under-weight one very strong signal.
Virtual Interviewer	Streams audio/video, transcribes live via Whisper, asks rubric-aligned questions.	Accuracy drops in noisy environments.
Scoring Engine	BERT-large rubric scorer plus a separate sentiment signal.	Ambiguous rubric wording adds scoring noise.
Governance Engine	Computes SR, ScR and Impact Ratio continuously; keeps demographic data isolated at the database level.	Small subgroup sizes make ratios unstable.

6. DATABASE DESIGN AND NORMALIZATION

6.1 ER Model

The schema is built around ten entity types. An Organization employs many Users through the Employs relationship. User specialises disjointly (the ‘d’ notation) into Admin, Recruiter and Auditor. A User creates Job_Postings (Creates), and the owning Organization also posts them (Posts). Each Job_Posting defines one or more Rubric_Criterion rows and is monitored by one or more Bias_Audit_Log rows. Candidates apply to Job_Postings through the M:N Applies relationship, resolved into the associative Application entity. A Candidate is decomposed into Proposition rows, a weak entity representing one atomic resume chunk. Each Application is optionally scheduled as one Interview_Session, which contains one or more Transcript rows, again a weak entity, this time for a single question-answer turn.

Fig. 6.1 Entity-Relationship Diagram (ECRAS)



6.2 Relational Schema

Applying the standard EER-to-relational mapping rules (Elmasri and Navathe, Ch. 9) to the diagram above gives one relation per strong entity, a mandatory foreign key on each weak entity back to its owner, a separate relation for the Applies M:N relationship, and a single Users table with a role discriminator column instead of three near-empty subtype tables, since none of Admin, Recruiter or Auditor has any attribute of its own. This gives ten relations:

Relation	Key Columns
Organizations	org_id (PK), org_name, org_domain, org_address
Users	user_id (PK), name, email, role, org_id (FK)
Job_Postings	job_id (PK), title, description, status, org_id (FK), created_by (FK)
Rubric_Criteria	criterion_id (PK), job_id (FK), criterion_text, weight
Bias_Audit_Logs	log_id (PK), job_id (FK), demographic_category, selection_rate, scoring_rate, impact_ratio
Candidates	candidate_id (PK), name, contact_email, phone, resume_doc_ref
Propositions	proposition_id (PK), candidate_id (FK), section, prop_text, embedding_ref
Applications	application_id (PK), candidate_id (FK), job_id (FK), applied_at, fused_score, rank
Interview_Sessions	session_id (PK), application_id (FK), started_at, ended_at, webrtc_room_ref
Transcripts	transcript_id (PK), session_id (FK), turn_index, question_text, answer_text, criterion_score

6.3 Functional Dependencies and Normalization

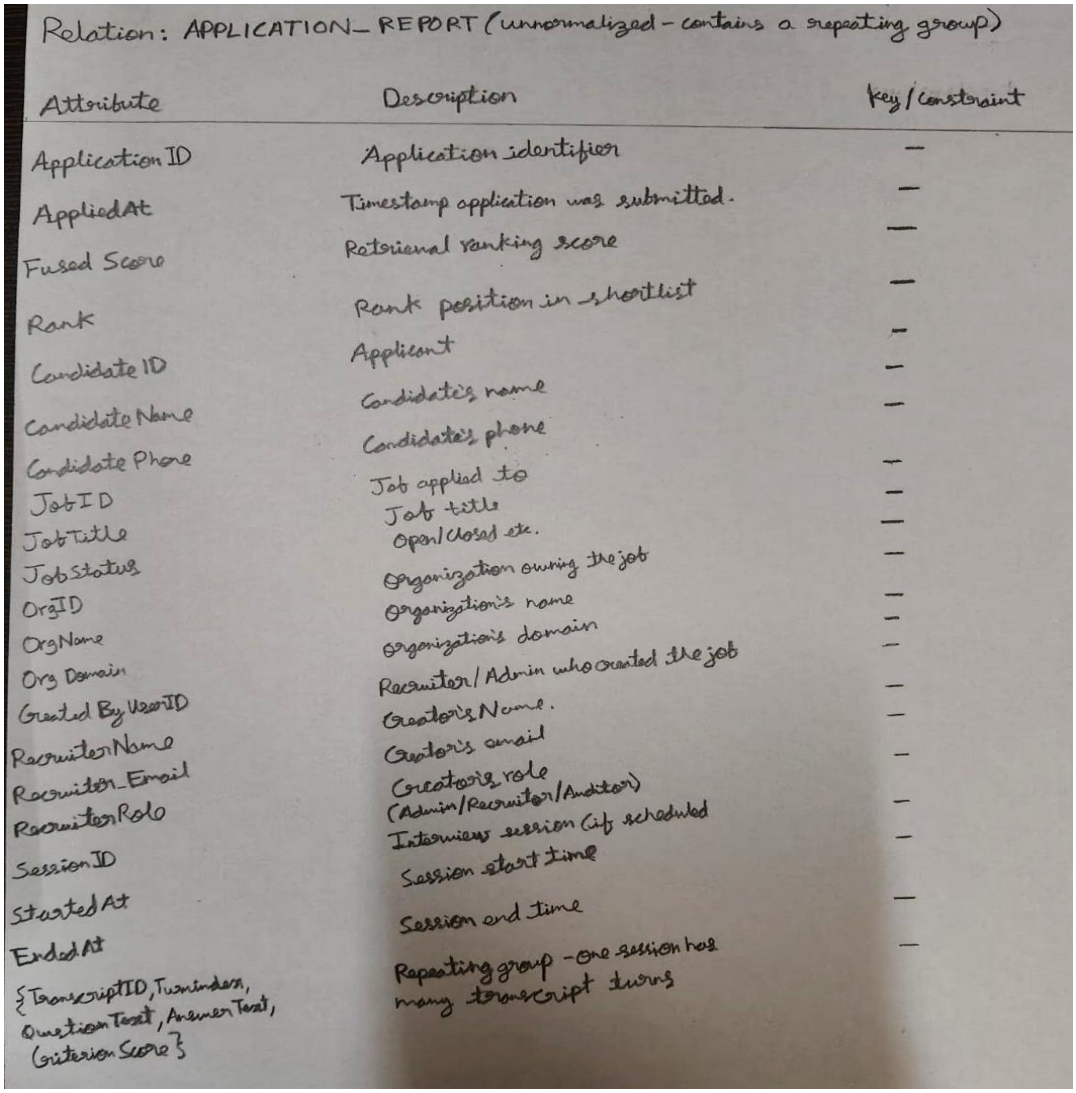
The ten relations above were mapped straight from the ER diagram rather than derived from a functional-dependency analysis, so this section works out that derivation independently and checks that the mapping is free of redundancy. Instead of normalizing all ten tables one by one, since most of them are already narrow, single-purpose relations, we start from the flat report a recruiter dashboard would actually query: one row per application, joined against Candidates, Job_Postings, Organization, the posting recruiter, the Interview_Session, and every Transcript turn. Rubric_Criteria, Bias_Audit_Logs and Propositions sit outside this flattened view since none of them is reachable from a single application record; they are checked separately in Section 6.3.6.

6.3.1 Unnormalized Form (UNF)

APPLICATION_REPORT (ApplicationID, AppliedAt, FusedScore, Rank, CandidateID, CandidateName, CandidateEmail, CandidatePhone, JobID, JobTitle, JobStatus, OrgID, OrgName, OrgDomain, CreatedByUserID, RecruiterName, RecruiterEmail, RecruiterRole, SessionID, StartedAt, EndedAt, ***{ TranscriptID, TurnIndex, QuestionText, AnswerText, CriterionScore }*** *)*

The braces mark a repeating group: one application's interview session can have many transcript turns, so this flat report violates 1NF.

Table 6.3a UNF



Relation: APPLICATION_REPORT (unnormalized – contains a repeating group)

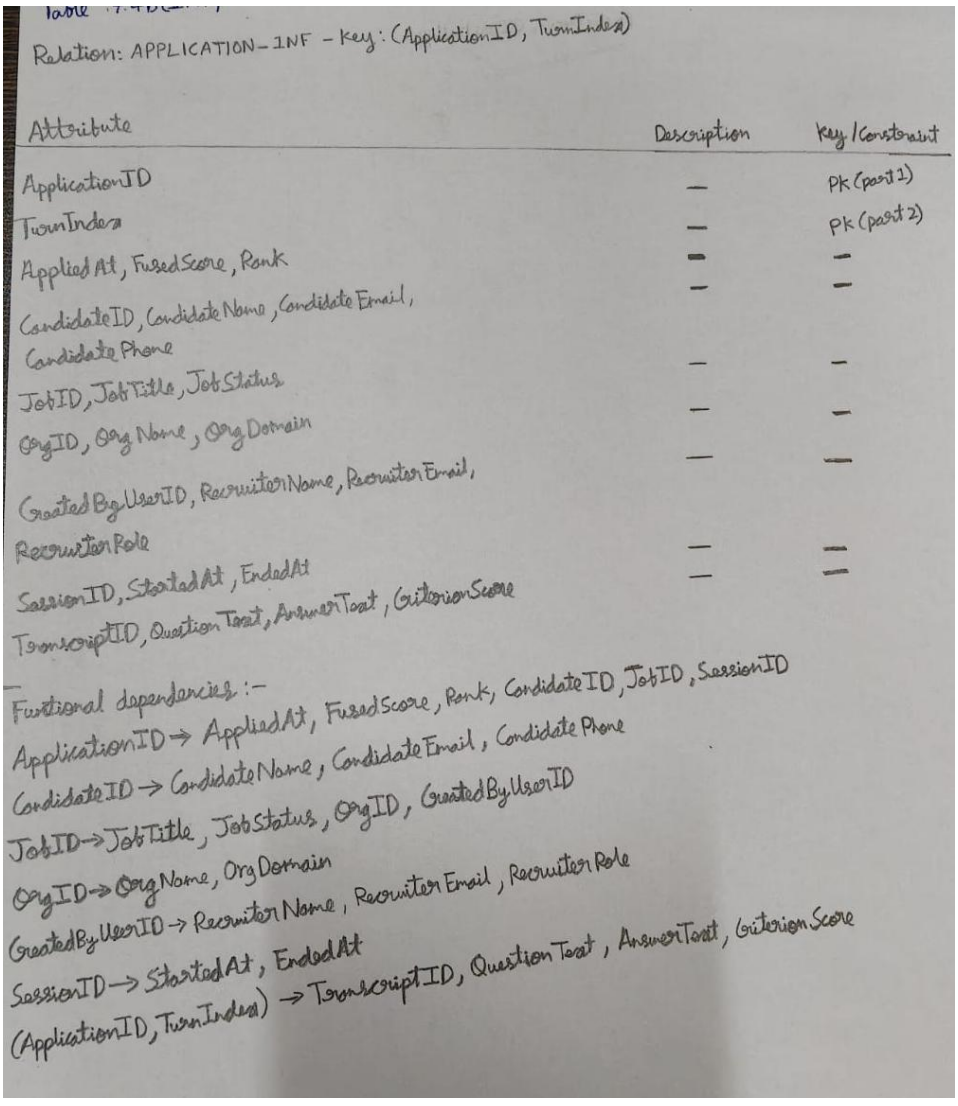
Attribute	Description	Key / Constraint
ApplicationID	Application identifier	—
AppliedAt	Timestamp application was submitted.	—
FusedScore	Retrieval ranking score	—
Rank	Rank position in shortlist	—
CandidateID	Applicant	—
CandidateName	Candidate's name	—
CandidatePhone	Candidate's phone	—
JobID	Job applied to	—
JobTitle	Job title	—
JobStatus	Open/closed etc.	—
OrgID	Organization owning the job	—
OrgName	Organization's name	—
OrgDomain	Organization's domain	—
CreatedByUserID	Recruiter/Admin who created the job	—
RecruiterName	Creator's Name.	—
RecruiterEmail	Creator's email	—
RecruiterRole	Creator's role (Admin/Recruiter/Auditor)	—
SessionID	Interview session (if scheduled	—
StartedAt	Session start time	—
EndedAt	Session end time	—
{TranscriptID, TurnIndex, QuestionText, AnswerText, CriterionScore}	Repeating group – one session has many transcript turns	—

6.3.2 First Normal Form (1NF)

Flattening the repeating group gives one row per (application, transcript turn), with composite key (ApplicationID, TurnIndex). The functional dependencies present at this stage are:

- ApplicationID → AppliedAt, FusedScore, Rank, CandidateID, JobID, SessionID
- CandidateID → CandidateName, CandidateEmail, CandidatePhone
- JobID → JobTitle, JobStatus, OrgID, CreatedByUserID
- OrgID → OrgName, OrgDomain
- CreatedByUserID → RecruiterName, RecruiterEmail, RecruiterRole
- SessionID → StartedAt, EndedAt
- (ApplicationID, TurnIndex) → TranscriptID, QuestionText, AnswerText, CriterionScore

Table 6.3b 1NF



Relation: APPLICATION_1NF – Key: (ApplicationID, TurnIndex)

Attribute	Description	Key / Constraint
ApplicationID	—	PK (part 1)
TurnIndex	—	PK (part 2)
AppliedAt, FusedScore, Rank	—	—
CandidateID, CandidateName, CandidateEmail, CandidatePhone	—	—
JobID, JobTitle, JobStatus	—	—
OrgID, OrgName, OrgDomain	—	—
CreatedByUserID, RecruiterName, RecruiterEmail, RecruiterRole	—	—
SessionID, StartedAt, EndedAt	—	—
TranscriptID, QuestionText, AnswerText, CriterionScore	—	—

Functional dependencies :-

ApplicationID → AppliedAt, FusedScore, Rank, CandidateID, JobID, SessionID

CandidateID → CandidateName, CandidateEmail, CandidatePhone

JobID → JobTitle, JobStatus, OrgID, CreatedByUserID

OrgID → OrgName, OrgDomain

CreatedByUserID → RecruiterName, RecruiterEmail, RecruiterRole

SessionID → StartedAt, EndedAt

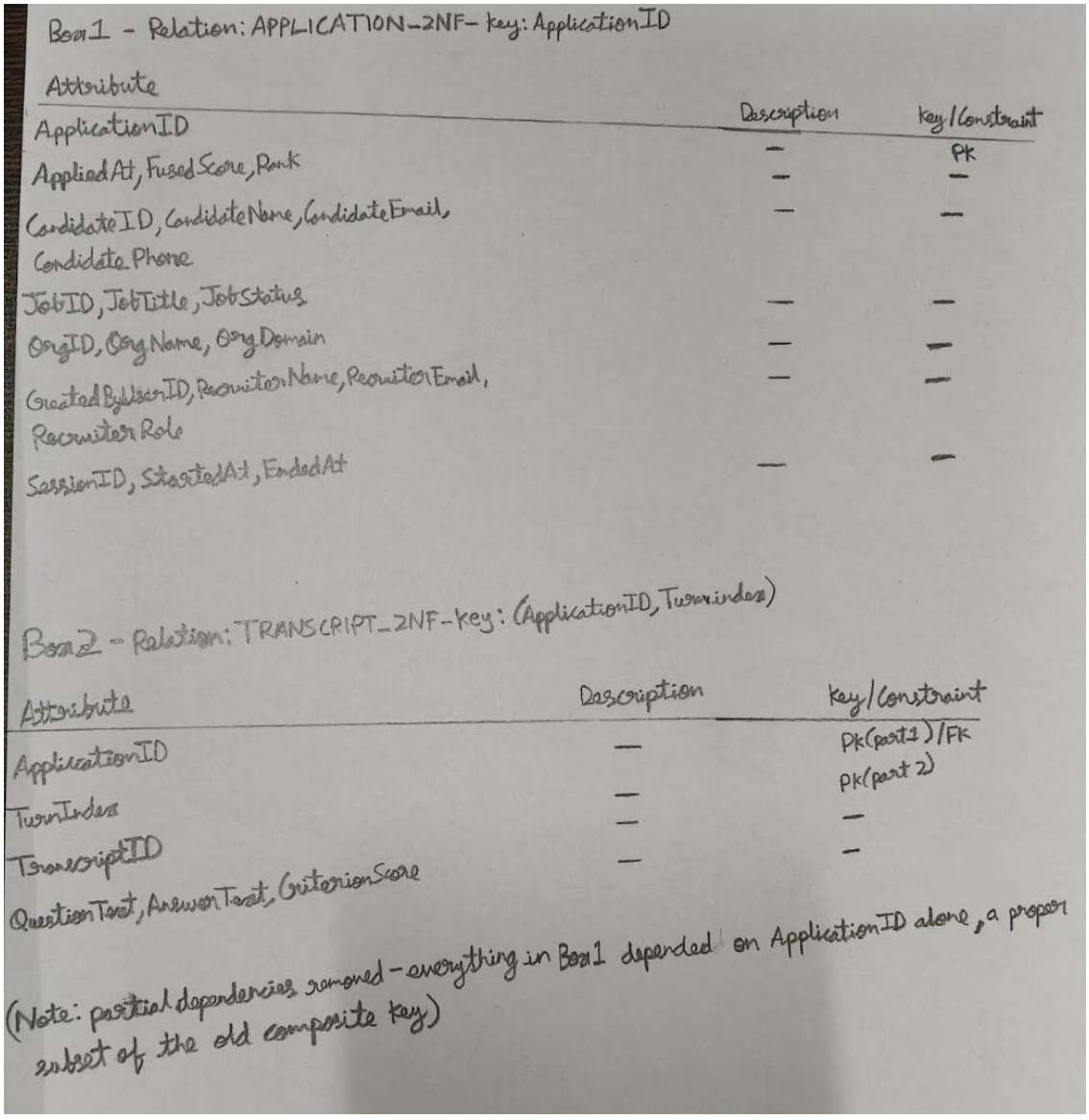
(ApplicationID, TurnIndex) → TranscriptID, QuestionText, AnswerText, CriterionScore

6.3.3 Second Normal Form (2NF)

Every attribute above depends on ApplicationID alone, a proper subset of the composite key, except the transcript-turn attributes, which genuinely need the full key. These are partial dependencies, and 2NF removes them by splitting the relation in two:

- APPLICATION_2NF (ApplicationID, AppliedAt, FusedScore, Rank, CandidateID, CandidateName, CandidateEmail, CandidatePhone, JobID, JobTitle, JobStatus, OrgID, OrgName, OrgDomain, CreatedByUserID, RecruiterName, RecruiterEmail, RecruiterRole, SessionID, StartedAt, EndedAt); Key: ApplicationID
- TRANSCRIPT_2NF (ApplicationID, TurnIndex, TranscriptID, QuestionText, AnswerText, CriterionScore); Key: (ApplicationID, TurnIndex)

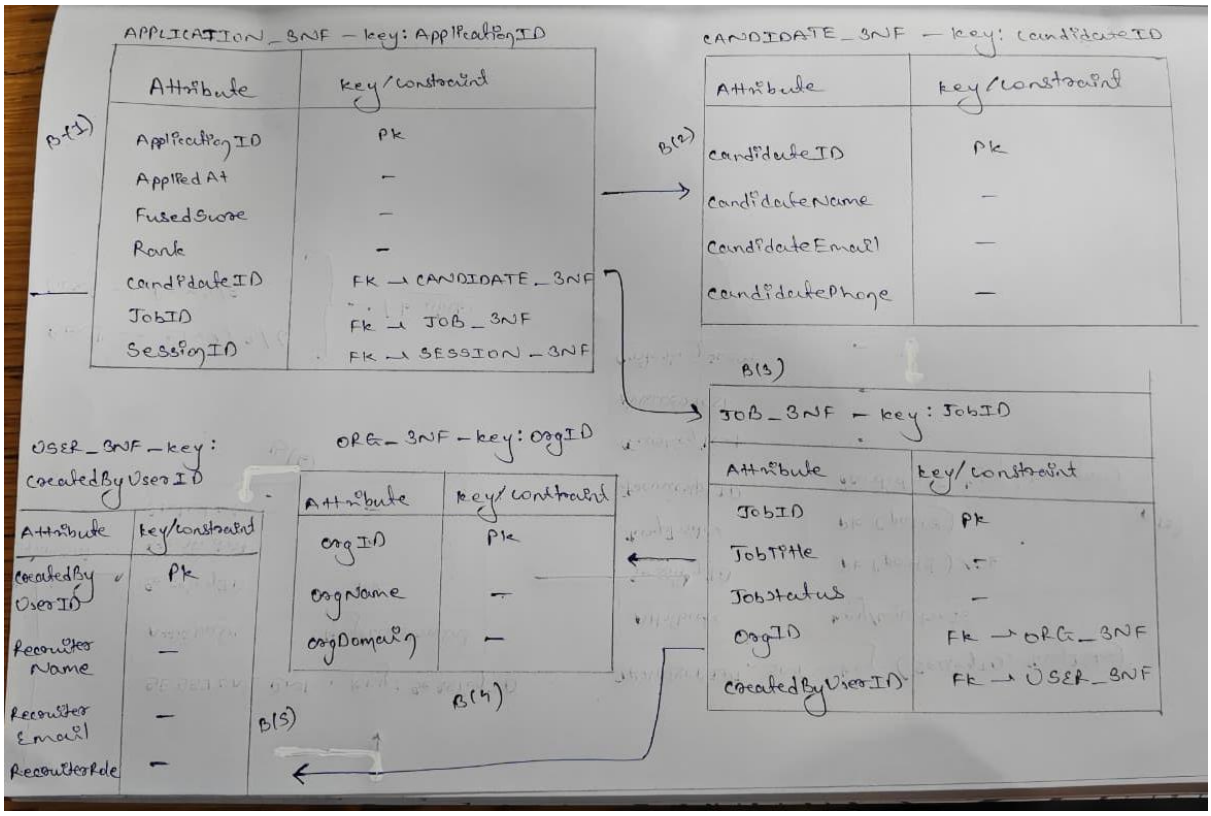
Table 6.3c 2NF

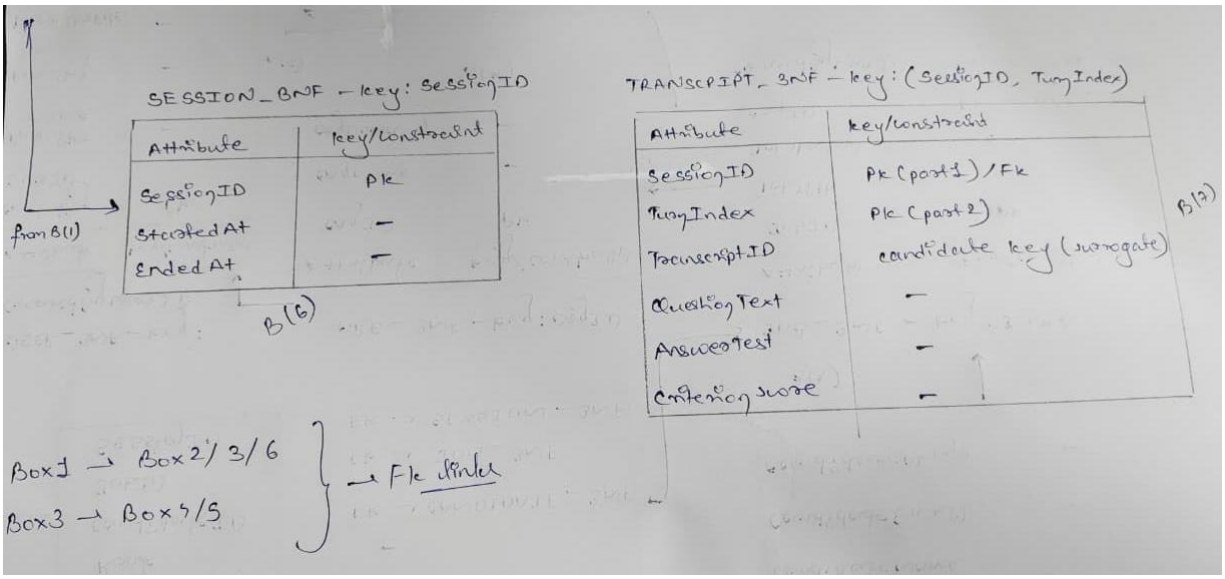


6.3.4 Third Normal Form (3NF)

APPLICATION_2NF still carries several transitive dependencies: ApplicationID → CandidateID → {Name, Email, Phone}; ApplicationID → JobID → {Title, Status, OrgID, CreatedByUserID}, with two further chains hanging off JobID itself (JobID → OrgID → {OrgName, OrgDomain} and JobID → CreatedByUserID → {RecruiterName, Email, Role}); and ApplicationID → SessionID → {StartedAt, EndedAt}. Each non-key determinant is projected out into its own relation:

Relation	Attributes	Key
APPLICATION_3NF	ApplicationID, AppliedAt, FusedScore, Rank, CandidateID(FK), JobID(FK), SessionID(FK)	ApplicationID
CANDIDATE_3NF	CandidateID, Name, Email, Phone	CandidateID
JOB_3NF	JobID, Title, Status, OrgID(FK), CreatedByUserID(FK)	JobID
ORG_3NF	OrgID, Name, Domain	OrgID
USER_3NF	CreatedByUserID, Name, Email, Role	CreatedByUserID
SESSION_3NF	SessionID, StartedAt, EndedAt	SessionID
TRANSCRIPT_3NF	SessionID(FK), TurnIndex, TranscriptID, QuestionText, AnswerText, CriterionScore	(SessionID, TurnIndex)





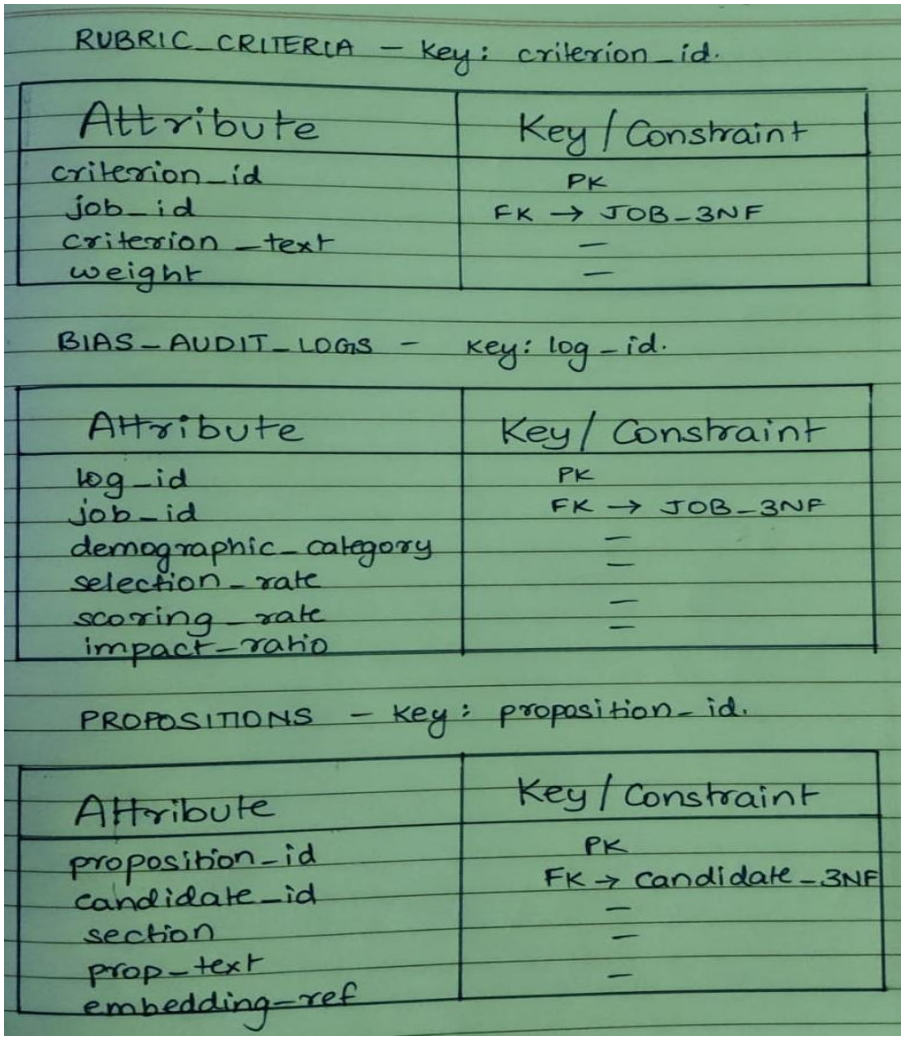
6.3.5 Boyce-Codd Normal Form (BCNF)

BCNF requires that for every non-trivial dependency X → Y, X must be a candidate key of its own relation. Checking each 3NF relation:

Relation	Non-trivial FD(s)	Determinant a key?	BCNF?
APPLICATION_3NF	ApplicationID → rest	Yes	Yes
CANDIDATE_3NF	CandidateID → rest	Yes	Yes
JOB_3NF	JobID → rest	Yes	Yes
ORG_3NF	OrgID → rest	Yes	Yes
USER_3NF	CreatedByUserID → rest	Yes	Yes
SESSION_3NF	SessionID → rest	Yes	Yes
TRANSCRIPT_3NF	(SessionID,TurnIndex) → rest; TranscriptID → rest	Yes, both	Yes

Every relation is already in BCNF: no determinant is a proper subset of, or disjoint from, its own candidate key. TRANSCRIPT_3NF happens to have two candidate keys, the natural composite (SessionID, TurnIndex) and the surrogate TranscriptID, and BCNF holds under either one.

Table 6.3e BCNF Verification



6.3.6 Rubric_Criteria, Bias_Audit_Logs, Propositions

These three tables sit outside the flattened application report because none of them is reachable from a single application row. Rubric_Criteria hangs off a Job_Posting, Bias_Audit_Logs is written by the governance engine independently of any one application, and Propositions hangs off a Candidate regardless of whether that candidate has applied anywhere. Each has exactly one non-trivial dependency, from its own surrogate key to everything else in the row:

- Rubric_Criteria: criterion_id → {job_id, criterion_text, weight}
- Bias_Audit_Logs: log_id → {job_id, demographic_category, selection_rate, scoring_rate, impact_ratio}
- Propositions: proposition_id → {candidate_id, section, prop_text, embedding_ref}

In each case the determinant is trivially the relation's sole candidate key, so all three are already in BCNF without any decomposition.

6.3.7 Lossless-Join and Dependency Preservation

The seven 3NF/BCNF relations above map directly onto Applications, Candidates, Job_Postings, Organizations, Users, Interview_Sessions and Transcripts from Section 6.2. Together with Rubric_Criteria, Bias_Audit_Logs and Propositions, this confirms, through explicit FD analysis rather than by assertion, that the finalised ten-relation schema is already in BCNF. Every split was made on a determinant that stays common to both resulting relations, for example CandidateID stays in both APPLICATION_3NF and CANDIDATE_3NF, which is the standard sufficient condition for a lossless-join decomposition. Dependency preservation also holds, since every FD identified back in Section 6.3.2 can still be checked inside a single resulting relation without needing to rejoin tables.

6.4 Constraints and Referential Integrity

Foreign-key constraints tie Users to Organizations; Job_Postings to both Organizations and Users (created_by); Rubric_Criteria and Bias_Audit_Logs to Job_Postings; Applications to Candidates and Job_Postings; Interview_Sessions to Applications; Transcripts to Interview_Sessions; and Propositions to Candidates. Cascading deletes are deliberately switched off on Bias_Audit_Logs so that audit history survives even if a job posting is withdrawn later, since this follows directly from the auditability requirement and not just from caution. A CHECK constraint keeps impact_ratio within the range [0, 1], and NOT NULL applies to every foreign key and to populated score/timestamp fields.

The disjoint specialisation of Users (Admin, Recruiter, Auditor) is enforced with a role CHECK constraint rather than three separate subtype tables, since none of the three roles carries a subtype-specific attribute in the ER diagram. Demographic data used for compliance is also kept in a schema separated from the scoring pipeline's read path at the database-permission level, not just by application convention, so that a bug in the ranking code cannot accidentally expose protected-category data.

One business rule worth noting: Job_Postings.org_id and the org_id of the recruiter in Job_Postings.created_by are expected to match, since a recruiter should not be able to post a job on behalf of an organization that does not employ them. This is a cross-relation business rule rather than a normalization issue, so it is enforced with a trigger or application-level check rather than a schema change.

7. TECHNOLOGY STACK

Layer	Technology	Role
Frontend	React.js, Material-UI, WebRTC	Recruiter dashboard, candidate interview portal
Backend API	Python 3.11, FastAPI, Node.js	Core orchestration, real-time services
Relational store	PostgreSQL	Organizations, Users, Candidates, Postings, Applications, Audit Logs
Cache / session	Redis	Session state, rate limiting, hot-path caching
Vector store	Qdrant (HNSW)	Dense proposition-embedding index
Speech-to-text	OpenAI Whisper v3	Real-time interview transcription
Scoring models	BERT-large, RoBERTa	Rubric scoring, sentiment signal

PostgreSQL and Redis are kept separate on purpose. Durable relational/audit data and short-lived session/cache data need different backup and retention policies, and an audit log should never be subject to the same eviction rules as a cache entry.

8. IMPLEMENTATION (SQL, PL/SQL AND APPLICATION)

The ten-relation BCNF schema derived in Section 6 has been implemented as a working system, with one supporting table, Audit_Trail, added for traceability (FR-10). This section documents that implementation in full: the Oracle SQL and PL/SQL database layer, the Node.js API that connects it to the interface, the React front end, and the tests used to check that each design rule from Sections 4 and 6 is actually enforced.

The database is the centre of the implementation. Keys, business rules, ranking, interview scoring and bias auditing are all enforced or computed inside Oracle; the API and the interface only call them. This keeps the rules in one place, so they hold no matter which client reaches the data.

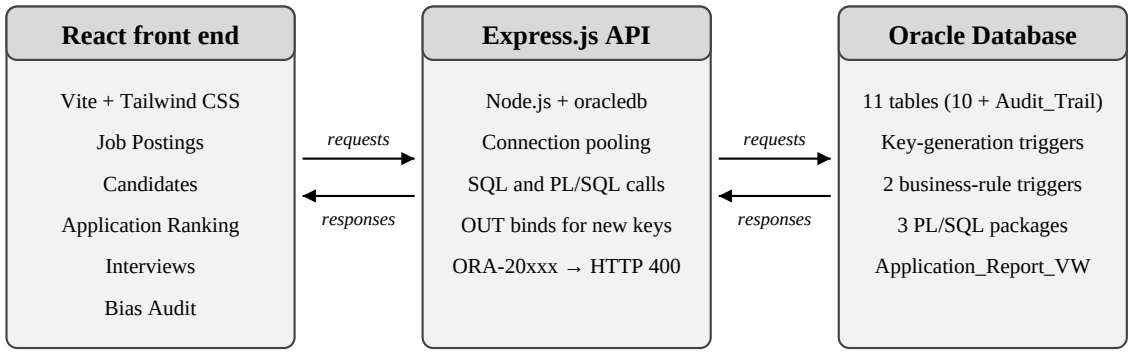
8.1 Implementation Overview and Architecture

The ECRAS platform is implemented as a three-tier web application that emphasizes strict database constraint enforcement and automated compliance auditing.

Table 8.1 Implementation tiers

Tier	Technology	Responsibility
Database Tier	Oracle SQL / PL/SQL	Schema, constraints, key generation, business-rule triggers, ranking, interview scoring and bias-audit logic
Backend Tier	Node.js with Express.js and the oracledb driver	Connection pooling, calling SQL and PL/SQL, translating Oracle errors into HTTP responses
Frontend Tier	React.js powered by Vite and Tailwind CSS	Recruiter dashboards for postings, candidates, ranking, interviews and bias audit

Fig. 8.1 Three-tier architecture of the ECRAS implementation



A user action in the React interface is sent to the Express API as an HTTP request. The API takes a connection from the oracledb pool and runs either a SQL statement or a PL/SQL call. Triggers and packages inside Oracle enforce the rules; the result, or an ORA-20xxx error raised by a rule, goes back to the API, which returns JSON or a readable HTTP 400 message

to the interface.

Table 8.2 Summary of implemented components

Component	What was implemented
Relations	The ten relations of Section 6.2, plus Audit_Trail
Key-generation triggers	One BEFORE INSERT trigger per table, backed by a sequence
Business-rule triggers	trg_job_org_match and trg_bias_audit_log_guard
PL/SQL packages	pkg_recruitment, pkg_interview and pkg_governance
View	Application_Report_VW, the flattened application report of Section 6.3.1
Interface modules	Job Postings, Candidates, Application Ranking, Interviews and Bias Audit, plus Privacy Policy, Terms and Conditions and a 404 page

8.2 Database Implementation (SQL)

The database layer was engineered to ensure data integrity, sequence management and automated compliance at the lowest possible level.

8.2.1 Schema Creation (DDL)

The DDL scripts create the ten relations of Section 6.2 in dependency order, from Organizations at the top of the foreign-key chain down to Transcripts, together with the Audit_Trail table. The scripts can be re-run, because existing objects are dropped before they are created. Every constraint stated in Section 6.4 is declared in the DDL rather than left to application code.

Table 8.3 Constraints implemented in the DDL

Constraint	Where it is applied	Purpose
PRIMARY KEY	Every relation (surrogate *_id keys)	Unique identity of each row; keys are generated by trigger
FOREIGN KEY	All relationships listed in Section 6.4	Referential integrity from Users up to Transcripts
CHECK	Users.role (ADMIN, RECRUITER, AUDITOR); Job_Postings.status (DRAFT, OPEN, CLOSED, WITHDRAWN); Bias_Audit_Logs.impact_ratio between 0 and 1	Enforces the disjoint specialisation of Users, the job life cycle, and a valid impact ratio
NOT NULL	Every foreign key and the populated score and timestamp fields	No orphan rows and no missing scores
UNIQUE	Applications (candidate_id, job_id)	A candidate can apply to a job only once
ON DELETE CASCADE	Weak entities: Interview_Sessions and Transcripts	Dependent rows are removed with their owner

Constraint	Where it is applied	Purpose
No cascade	Bias_Audit_Logs → Job_Postings	Audit history survives (Section 6.4); a posting with audit rows cannot be deleted

8.2.2 Sequences and Automated Key Generation

BEFORE INSERT triggers were implemented on all tables. If a primary key is NULL during an INSERT, the trigger fetches the next sequence value automatically, so neither the PL/SQL packages nor the API need to know any sequence name. The sequences start at 1000, which keeps generated keys apart from the seeded rows: in Fig. 8.2 the seeded postings carry IDs 1 to 5, while the postings added afterwards received IDs 1000 and 1001. The API uses Oracle OUT bind variables to read the generated key straight back from the INSERT.

8.2.3 Seed Data

A seed script loads realistic demonstration data so that every query, package and screen has something meaningful to work on: three organizations (Nimbus Analytics, Solstice Health Systems and Vertex Manufacturing Co), users in the Admin, Recruiter and Auditor roles, job postings in the OPEN, DRAFT and CLOSED states, candidates spread across several demographic categories, applications with fused scores and ranks for each job, interview sessions with several transcript turns each, and sample Bias_Audit_Logs rows.

8.2.4 Core SQL Operations

The operations required for the project, creation, insertion, updating, deletion and retrieval, are each implemented and each is reachable from the application.

Table 8.4 SQL operations and where they are used

Operation	Implementation in ECRAS	Used by
CREATE	CREATE TABLE, SEQUENCE, TRIGGER, VIEW and PACKAGE statements in the build scripts	Database set-up
INSERT	New Job_Postings and Candidates rows (key left NULL, trigger assigns it); Applications through sp_insert_application; Transcripts through sp_insert_transcript_turn; Bias_Audit_Logs through sp_compute_bias_audit	Create Job Posting, Add Candidate, Apply to Job, Add Turn, Compute Audit
UPDATE	rank_position rewritten for every application of a job by sp_rerank_applications; an existing Bias_Audit_Logs row is updated, not duplicated, when an audit is recomputed	Re-rank Candidates, Compute Audit
DELETE	Deliberately constrained: deletes on Bias_Audit_Logs are rejected by trigger, a posting that has audit rows is protected by its foreign key, and Transcripts and Interview_Sessions are removed with their parent	Compliance rules (see Section 8.7)

Operation	Implementation in ECRAS	Used by
SELECT / JOIN	Application_Report_VW joins Applications, Candidates, Job_Postings, Organizations, Users and Interview_Sessions; transcript retrieval; aggregate rates for the bias audit	Application Ranking, Interviews, Bias Audit

8.2.5 The Application_Report_VW View

Application_Report_VW rebuilds the flat APPLICATION_REPORT of Section 6.3.1 by joining the normalized relations back together. It closes the loop with the normalization work: the design starts from the unnormalized report, decomposes it to BCNF, and the view shows that the decomposition loses nothing. The Application Ranking screen reads from this view, so each row it shows (rank, candidate, demographic label and fused score) comes from a single join over the BCNF tables.

8.2.6 Schema Reconciliation and Normalization Decisions

Table 8.5 Reconciliation decisions made during implementation

Issue	Resolution
Keyword collision: the RANK column of Applications clashes with Oracle's reserved analytical functions	Renamed to rank_position across all DDL, PL/SQL packages and views
Primary keys tied to sequence names	BEFORE INSERT triggers fill a NULL key, so business logic is decoupled from sequence management
Enum states written in different cases	Enum states (for example job statuses) are strictly normalized; the Express backend and the Oracle constraints both enforce uppercase formats such as 'OPEN'
The bias audit needs a group for every candidate	A demographic_category column was added to Candidates, so joins across Applications, Candidates and Interview_Sessions can compute the EEOC bias metrics accurately. The column is kept on Candidates for simplicity, whereas Section 6.4 describes isolating demographic data at the database-permission level; moving it into a separately permissioned schema is left as future work
FR-10 needs a trace of automated decisions and overrides	Audit_Trail table, written by trg_bias_audit_log_guard through autonomous transactions

8.3 PL/SQL Implementation

8.3.1 Business Logic Packages

The project-specific logic is grouped into three packages, one per module of Section 5. Together they hold everything that ranks, scores and audits, so none of it depends on the API or the interface.

Table 8.6 PL/SQL packages and subprograms

Package	Subprogram	What it does
pkg_recruitment	sp_insert_application	Validates that the candidate and the job exist, computes the fused score from the lexical and semantic scores supplied with the application, inserts the Applications row and re-ranks the job
	sp_rerank_applications	Recalculates the dynamic leaderboard: a cursor loop orders the applications of one job by fused_score, highest first, and writes rank_position 1, 2, 3 …
pkg_interview	fn_compute_interview_score	Calculates the average criterion_score over all transcript turns of a session, using a cursor, every time it is called
	sp_insert_transcript_turn	Stores a new question-and-answer turn under the next turn_index of the session and returns the recalculated score
pkg_governance	sp_compute_bias_audit	Runs entirely inside the Oracle database: computes the selection rate, scoring rate and impact ratio of a demographic category for a job, flags groups that fall below the EEOC 80% (four-fifths) threshold, and stores the result in Bias_Audit_Logs

The lexical and semantic inputs to sp_insert_application are simulated scores that stand in for the BM25 and dense-retrieval scores of Section 3.1; the fusion, ranking and storage that follow are real database logic. Every subprogram has exception handling, and rule violations are raised with RAISE_APPLICATION_ERROR so that callers receive a clear, numbered message.

8.3.2 Active Business Rule Enforcement (Triggers)

Table 8.7 Triggers implemented

Trigger	Fires on	Rule and behaviour
trg_job_org_match	INSERT on Job_Postings	Validation trigger ensuring that users can only create job postings for the organization they belong to (the cross-relation rule of Section 6.4). A mismatch raises ORA-20001
trg_bias_audit_log_guard	UPDATE and DELETE on Bias_Audit_Logs	Compliance trigger that blocks all DELETE operations with ORA-20010. Attempted changes are caught silently and recorded in Audit_Trail (table, operation, row key, old values, user, time) using an autonomous transaction, so the record survives even when the statement itself is rejected
Key-generation triggers	INSERT on every table	Assign the next sequence value when the primary key is NULL

Autonomous transactions matter for the guard trigger. An ordinary log write would be rolled back together with the rejected DELETE; running the log write in its own transaction means the attempt is still recorded, which is exactly the traceability that FR-10 asks for.

8.4 Backend (API) Integration

- **Oracle connectivity:** the official oracledb library is used for high-performance connection pooling, so requests share a pool of ready connections instead of opening a new one each time.
- **Error handling translation:** the API intelligently catches the custom Oracle application exceptions (ORA-20xxx) thrown by the database triggers and packages and translates them into clean, human-readable HTTP 400 Bad Request responses for the frontend.
- **OUT binds:** Oracle OUT bind variables immediately capture the sequence-generated primary keys produced by the triggers and return them in the HTTP response body, without a secondary SELECT query.
- **Enum validation:** the backend enforces the same uppercase formats as the Oracle constraints, so invalid values are rejected before they reach the database.

Each action in the interface maps to one database object, which keeps the API thin:

Table 8.8 Interface actions and the database objects behind them

Interface action	Database object invoked
Create Posting (Fig. 8.3)	INSERT into Job_Postings, checked by trg_job_org_match
Save candidate (Fig. 8.5)	INSERT into Candidates with demographic_category
Apply to Job (Fig. 8.4)	pkg_recruitment.sp_insert_application
Choose a job on Application Ranking (Fig. 8.7)	SELECT from Application_Report_VW
Re-rank Candidates (Fig. 8.7)	pkg_recruitment.sp_rerank_applications
Load Session (Fig. 8.9)	SELECT from Transcripts and pkg_interview.fn_compute_interview_score
Add Turn (Fig. 8.9)	pkg_interview.sp_insert_transcript_turn
Compute Audit (Fig. 8.11)	pkg_governance.sp_compute_bias_audit, then SELECT from Bias_Audit_Logs

8.5 Frontend and User Interface

The frontend provides an interactive dashboard to the PL/SQL engine. This section walks through each screen of the running application.

8.5.1 Layout and Navigation

A fixed left sidebar lists the five modules (Job Postings, Candidates, Application Ranking, Interviews and Bias Audit) with the active one highlighted, and links to the Privacy Policy and Terms and Conditions pages at its foot. Every module follows the same pattern: a title with a one-line description, a primary action button at the top right, and card or table content below, so a recruiter can move between modules without relearning the layout.

8.5.2 Job Postings Dashboard

Fig. 8.2 Job Postings dashboard

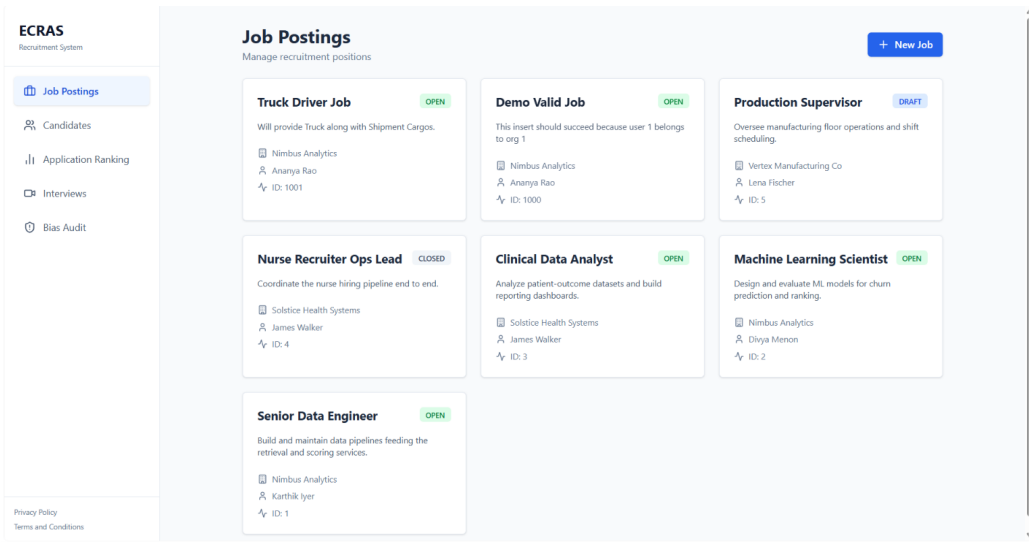
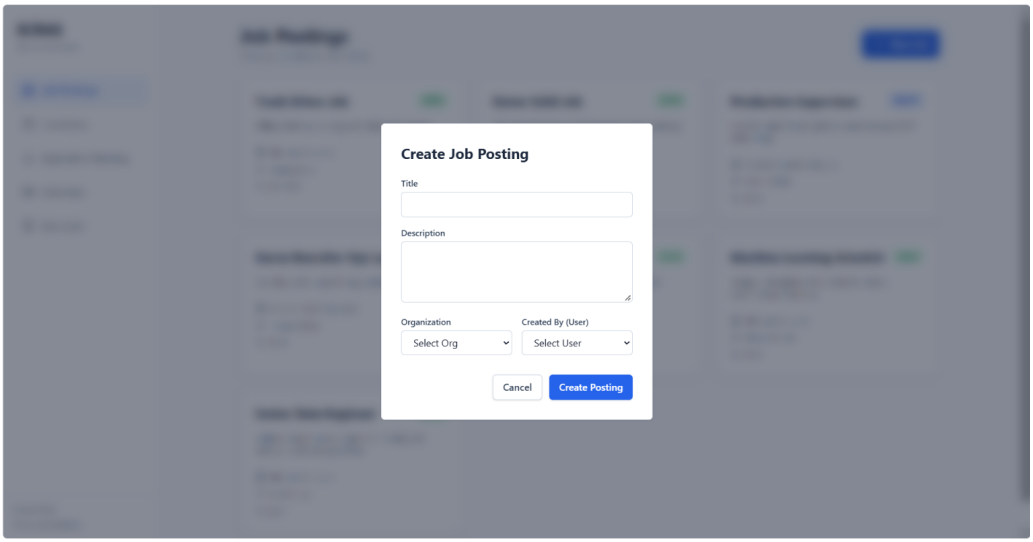


Fig. 8.2 is the entry point of the application, where recruitment positions are viewed and managed. Each row of the Job_Postings relation appears as a card with the job title, a status badge (OPEN, DRAFT or CLOSED), the description, the owning organization, the user who created the posting and the job_id. The seven postings shown belong to three organizations, and each is tied to exactly one organization and one creating user, which is the pairing that trg_job_org_match validates. The New Job button at the top right opens the form in Fig. 8.3.

Fig. 8.3 Create Job Posting form



The Create Job Posting form collects the Title, the Description, the Organization and the Created By (User). Both drop-down lists are filled from the Organizations and Users relations, so only existing keys can be chosen and the NOT NULL foreign keys can never be left empty. On Create Posting the API inserts the row; the trigger assigns the job_id and it is returned through an OUT bind, and the new card appears on the dashboard with status OPEN. If the selected user does not belong to the selected organization, trg_job_org_match raises ORA-20001; the API returns it as an HTTP 400 and the form shows the rule that was broken instead of an Oracle error stack.

8.5.3 Candidates Dashboard

Fig. 8.4 Candidates dashboard

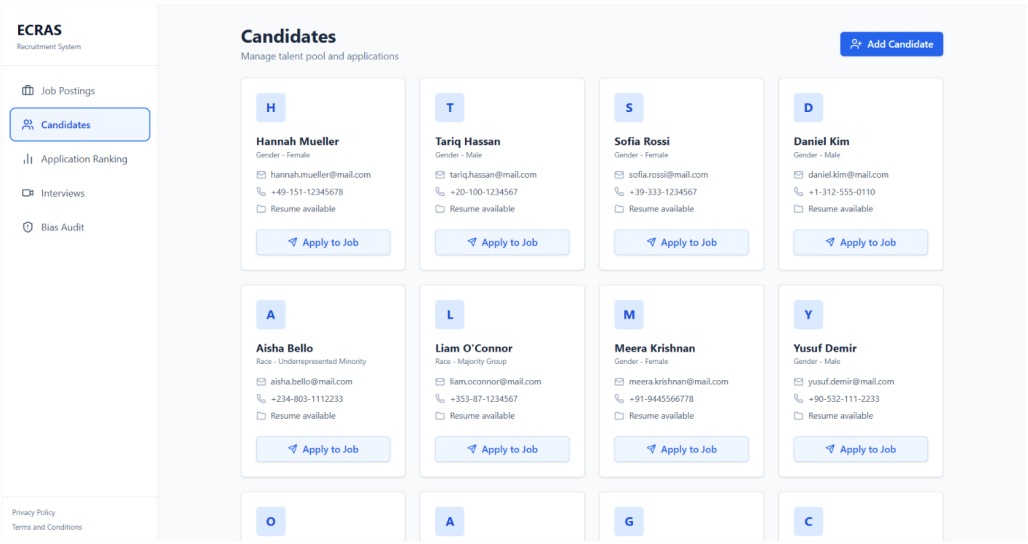
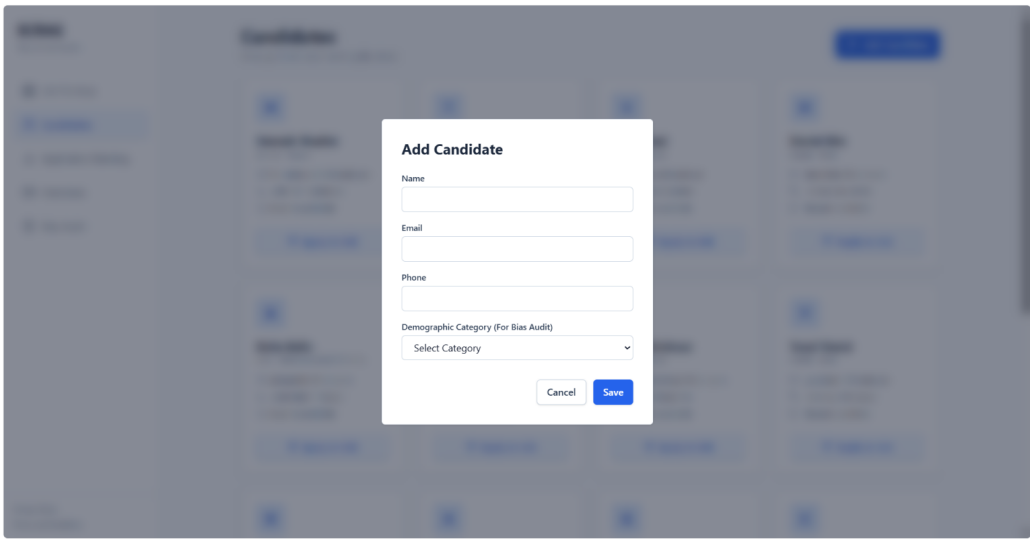


Fig. 8.4 shows the talent pool, which holds every candidate and is the starting point for applications. Each Candidates row appears as a card with an initial avatar, the name, the

demographic category label, the email address, the phone number, a resume indicator and an Apply to Job button. Labels such as Gender - Female or Race - Underrepresented Minority come from the demographic_category column and are what the bias audit later groups by. Apply to Job submits the candidate to a posting through pkg_recruitment.sp_insert_application, which stores the application with its fused score and re-ranks that job.

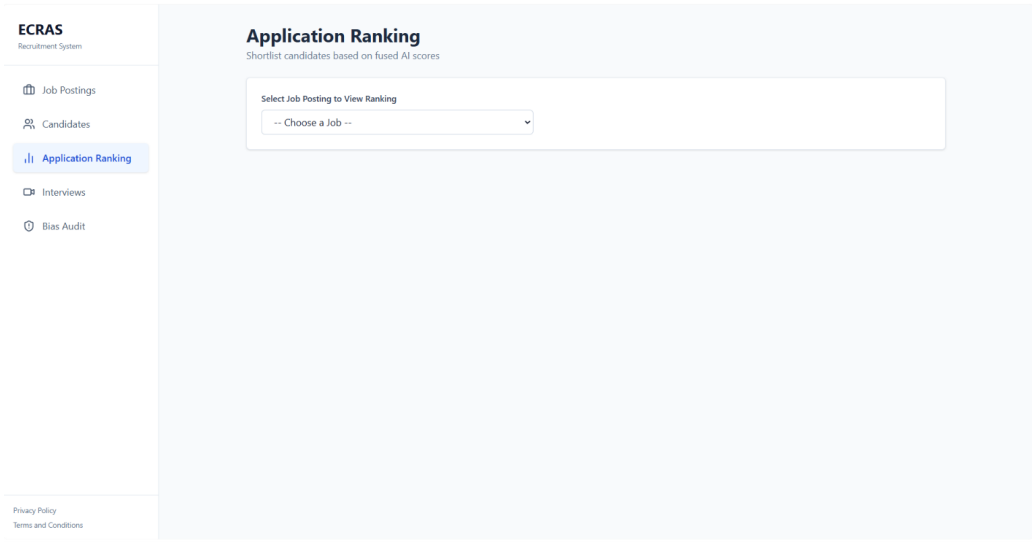
Fig. 8.5 Add Candidate form



The Add Candidate form takes the Name, Email and Phone, and a Demographic Category (For Bias Audit) drop-down that offers the same categories the audit reports on. Save inserts a Candidates row; the candidate_id is generated by the key trigger and returned to the page through an OUT bind. Keeping the category on the form means every new candidate is included in the bias audit from the moment they are added.

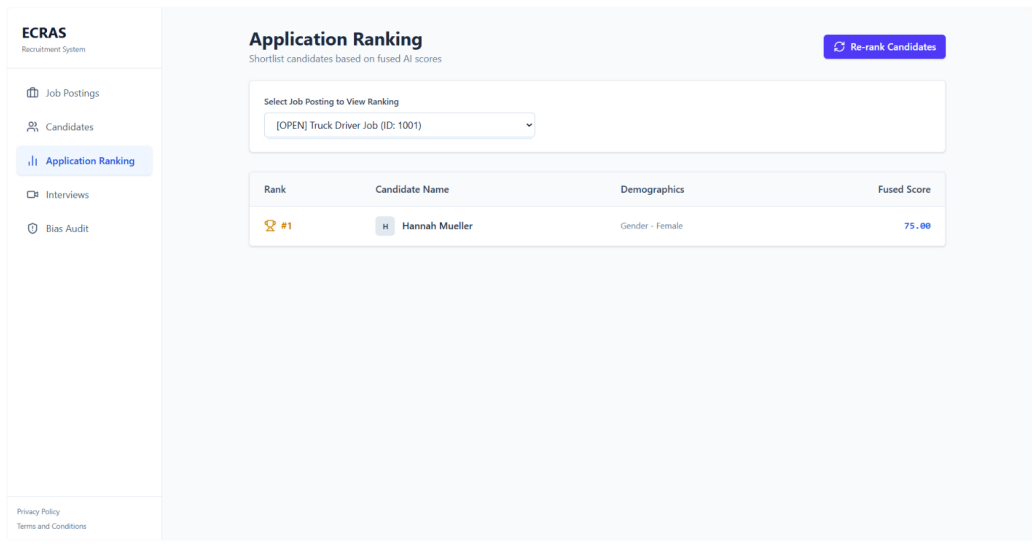
8.5.4 Application Ranking Dashboard

Fig. 8.6 Application Ranking dashboard (job selection)



The Application Ranking module shortlists candidates based on fused AI scores. It starts with a job selector, whose options show the status, the title and the ID of each posting, for example [OPEN] Truck Driver Job (ID: 1001). Nothing is listed until a job is chosen, because ranks only have meaning within one posting.

Fig. 8.7 Ranked applicants for the Truck Driver Job

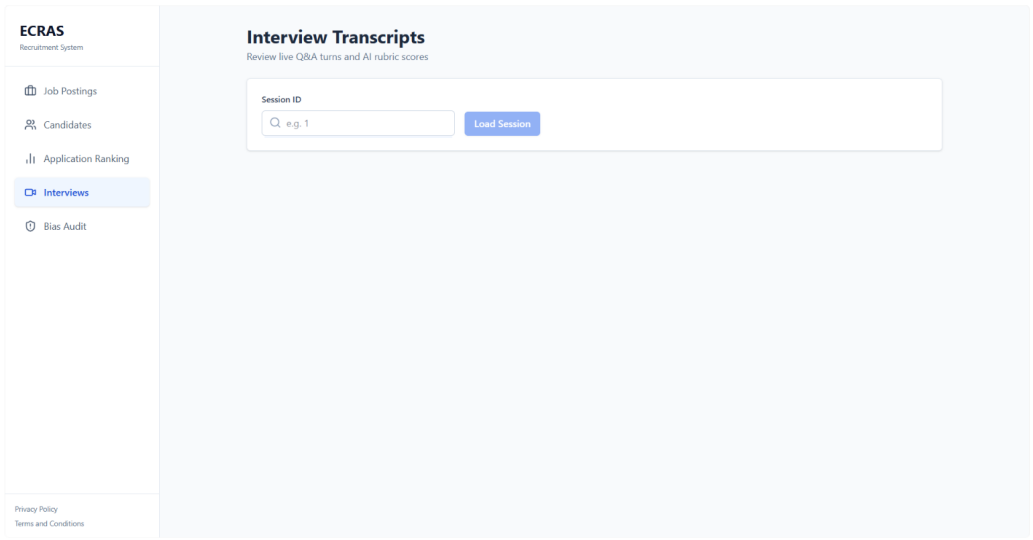


Once the Truck Driver Job is selected, Fig. 8.7 lists its applicants in rank order. The table has four columns: Rank (the stored rank_position, with a trophy icon for #1), Candidate Name, Demographics and Fused Score. The Re-rank Candidates button calls sp_rerank_applications, which orders the applications of the job by fused_score from highest to lowest and rewrites rank_position inside the database, so what the screen shows always matches the stored values. Here one applicant has applied, Hannah Mueller, with a fused score of 75.00, at rank #1; as

more applicants are added and the job is re-ranked, the order adjusts automatically. Ranks are assigned per job, so every position has its own #1. The list is a shortlist for the recruiter; as Section 1.3 states, the final hiring decision stays with a human.

8.5.5 Interview Transcripts Dashboard

Fig. 8.8 Interview Transcripts dashboard (session lookup)



The Interview Transcripts module reviews live question-and-answer turns together with the AI rubric scores. It is keyed on the interview session. The recruiter types a Session ID, the session_id of an Interview_Sessions row, which belongs to one application, and presses Load Session. The button stays greyed out until an ID has been entered, which prevents empty requests from reaching the API.

Fig. 8.9 Transcript turns and session summary for Session ID 2

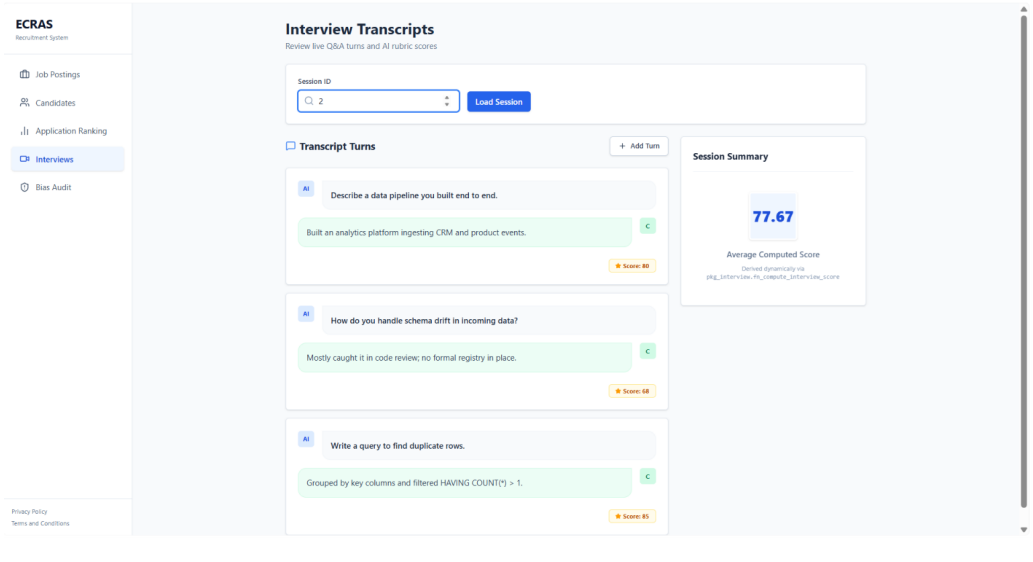
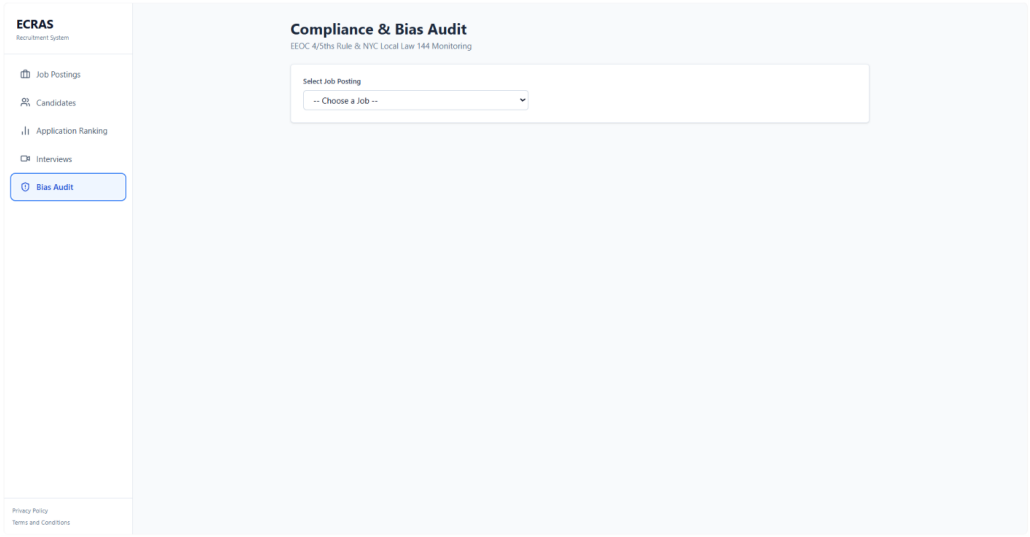


Fig. 8.9 shows Session ID 2 after loading. The page has two parts. On the left, Transcript Turns lists each Transcripts row as an exchange: the AI interviewer's question (AI avatar), the candidate's answer (C avatar) and the rubric score awarded to that answer in a badge. The session has three turns: a question on building a data pipeline end to end (score 80), one on handling schema drift (score 68), and one asking for a query that finds duplicate rows (score 85). Because every score sits next to the exact question and answer that produced it, the evidence behind each number can be read directly, as Section 3.2 requires.

On the right, the Session Summary shows an Average Computed Score of 77.67, labelled as derived dynamically via pkg_interview.fn_compute_interview_score. The value is not stored anywhere: the function averages the criterion_score of all turns each time the page loads, so it can never disagree with the transcript. The figure can be checked by hand: (80 + 68 + 85) / 3 = 77.67. The Add Turn button appends a new question-and-answer turn by calling sp_insert_transcript_turn, which stores it under the next turn_index of the session and recalculates the summary, so a new score changes the average straight away. Deriving the average instead of storing it also avoids an update anomaly, in keeping with the BCNF design of Section 6.3.

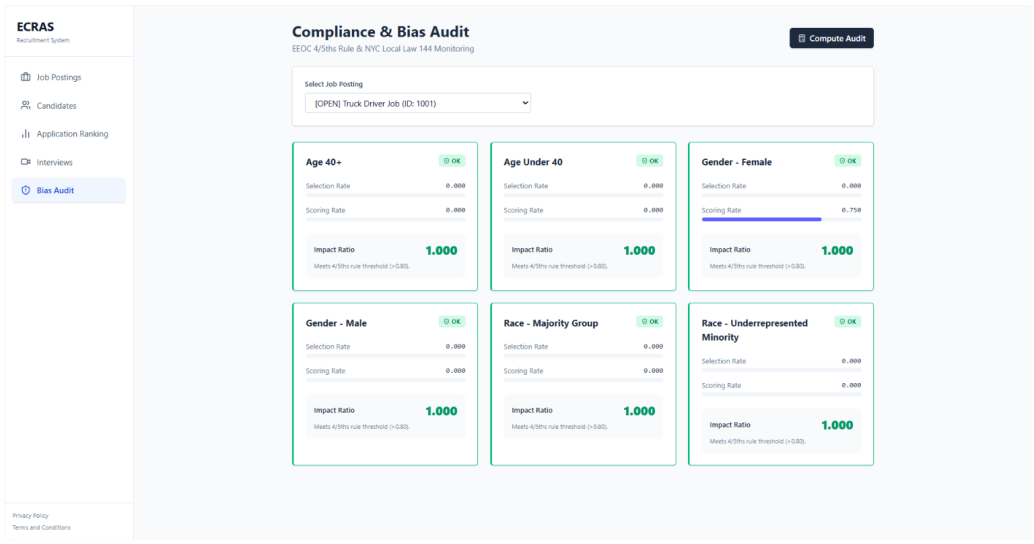
8.5.6 Compliance and Bias Audit Dashboard

Fig. 8.10 Compliance and Bias Audit dashboard (job selection)



The Compliance and Bias Audit module monitors the EEOC four-fifths rule and NYC Local Law 144, the two regulations that motivated the project in Section 1.1. As on the ranking screen, the recruiter or auditor first selects a job posting.

Fig. 8.11 Computed bias audit for the Truck Driver Job (ID 1001)



For the chosen job, the Compute Audit button runs pkg_governance.sp_compute_bias_audit inside Oracle and the page then shows one card for each demographic category: Age 40+, Age Under 40, Gender - Female, Gender - Male, Race - Majority Group and Race - Underrepresented Minority. Each card reports three measures from Section 4.1 (FR-8):

- **Selection Rate:** the share of the group's applicants who were selected.
- **Scoring Rate:** the group's average fused score, expressed between 0 and 1.
- **Impact Ratio:** the group's selection rate divided by the highest selection rate among the groups. Under the four-fifths rule a ratio below 0.80 signals possible adverse impact.

A card carries an OK badge and a green outline, with the note "Meets 4/5ths rule threshold (>0.80)", when the ratio passes. A group that violates the rule is flagged in red. In Fig. 8.11 all six groups pass. Only one applicant has applied to this job so far, Hannah Mueller (Gender - Female, fused score 75.00), which is why Gender - Female shows a scoring rate of 0.750, no group has a selection rate above 0.000, and every impact ratio is 1.000. The result is correct for the data present, but it also shows the limitation noted for the Governance Engine in Section 5: ratios from very small groups are unstable. Each result is saved in Bias_Audit_Logs, whose impact_ratio is constrained to the range 0 to 1, and which cannot be deleted from, so the audit history is preserved for Local Law 144 disclosure (FR-9).

8.6 UI/UX and Engineering Standards

Table 8.9 UI/UX and engineering standards

Area	Implementation
Professional aesthetics	The UI was refactored to strict professional design standards: a restrained colour palette, border radii of exactly 4px to 6px, and no unnecessary gradients or animations
Accessibility	Web Content Accessibility Guidelines (WCAG) enforced with strict semantic HTML (tracked <h1> hierarchy), active keyboard focus states (focus:ring-2) and mobile-responsive grid layouts
Compliance pages	Mandatory legal scaffolding: a Privacy Policy, Terms and Conditions, and a custom 404 Not Found page
PWA and meta readiness	A complete suite of application icons (favicon.ico, apple-touch-icon, manifest.json) and Open Graph meta tags, so the application renders properly when shared or installed

8.7 Testing and Verification

Testing was done at two levels. The PL/SQL layer was exercised directly in SQL with a numbered series of test blocks that print their results, and the running application was then checked against the figures the database should produce.

Table 8.10 Database-level tests of the PL/SQL layer

No.	Test	Observed result
1	Insert a job posting whose organization matches its creator	1 row inserted
2	Insert a job posting for an organization the creator does not belong to	Rejected: ORA-20001, “Business rule violated: job posting org_id must match the org_id of the user who created it.”
3	Add an application with sp_insert_application	Fused score of 81.5 calculated; 5 applications re-ranked for the job
4	List the ranked applications of the job	Fused scores 91, 82, 81.5, 74 and 67 hold ranks 1 to 5
5	Add a transcript turn with sp_insert_transcript_turn	Turn 3 stored for the session; computed interview score updated to 4.13
6	Compute the bias audit for two groups	Group A: selection rate 1, scoring rate 0.865, impact ratio 1. Group B: selection rate 0, scoring rate 0.742, impact ratio 0
7	Delete a row from Bias_Audit_Logs	Rejected: ORA-20010, “Bias_Audit_Logs rows cannot be deleted for audit traceability.”
8	Delete a job posting that has audit rows	Rejected: ORA-02292, integrity constraint violated, child record found
9	Inspect Audit_Trail	The UPDATE and the blocked DELETE on Bias_Audit_Logs are both recorded, with old values, user and timestamp

The sample figures in Table 8.10 come from the test data used while the packages were developed, so they differ from the demonstration data shown in the screenshots; what the table establishes is that each rule behaves as designed. The application was then checked against the database, as summarised in Table 8.11.

Table 8.11 Application-level verification

No.	Check in the running application	Result
1	Session 2 in Fig. 8.9 has turn scores 80, 68 and 85	The Session Summary shows 77.67, equal to (80 + 68 + 85) / 3, computed by fn_compute_interview_score
2	Ranking of the Truck Driver Job in Fig. 8.7	The applicant shown holds rank #1 with fused score 75.00; ranks are assigned within the job
3	Keys of rows created after the seed data was loaded	Job postings received IDs 1000 and 1001, from the sequence through the key trigger, apart from the seeded IDs 1 to 5
4	Bias audit of the Truck Driver Job in Fig. 8.11	Scoring rate 0.750 for Gender - Female matches the single applicant’s 75.00 fused score; every impact ratio is 1.000 and all six cards show OK
5	Job creation with mismatched organization and user	The trigger error is shown on the form as a readable HTTP 400 message

8.8 Traceability to Design Requirements

The table below maps the requirements and constraints stated earlier in this report to the objects and screens that implement them.

Table 8.12 Requirement traceability

Requirement (source)	Implemented by	Shown in
Job posting org_id must match its creator’s org_id (Section 6.4)	trg_job_org_match	Fig. 8.3
Audit history must survive; no deletes on Bias_Audit_Logs (Section 6.4)	trg_bias_audit_log_guard; no cascade from Job_Postings	Table 8.10, tests 7 and 8
Role, status and impact-ratio domains (Section 6.4)	CHECK constraints on Users, Job_Postings and Bias_Audit_Logs	Table 8.3
Hybrid ranking of applicants (FR-3)	pkg_recruitment: fused score from simulated lexical and semantic inputs, then re-ranking. The BM25 + dense + RRF pipeline of Section 3.1 lies outside the database layer	Fig. 8.7
Score every answer against a rubric (FR-6)	Transcripts.criterion_score stored with each question and answer	Fig. 8.9
Scorecard per candidate (FR-7)	Transcript turns with their scores and the derived session average	Fig. 8.9

Requirement (source)	Implemented by	Shown in
Selection rate, scoring rate and impact ratio per group (FR-8)	pkg_governance.sp_compute_bias_audit	Fig. 8.11
Bias-audit report for Local Law 144 disclosure (FR-9)	Stored, undeletable Bias_Audit_Logs rows shown per category	Fig. 8.11
Log automated decisions and overrides (FR-10)	Audit_Trail table, written through autonomous transactions	Table 8.10, test 9

8.9 Limitations and Assumptions

- The lexical and semantic match scores are simulated inputs. The retrieval and reranking components of Section 3 (BM25, Qdrant, the cross-encoder) and the interview components (WebRTC, Whisper, the scoring models) are not part of this database implementation; their outputs are represented by data entered through the interface.
- demographic_category is stored on Candidates and holds one value per candidate, so each candidate belongs to a single group. Section 6.4 describes a separately permissioned schema, which is left as future work.
- The posting user is chosen explicitly on the Create Job Posting form. The OAuth2 and role-based access control of Section 4.2 are not part of this implementation.
- Bias ratios depend on subgroup size. With few applicants, as in Fig. 8.11, every group can pass or fail by chance, so results should be read together with the number of applicants behind them.

9. CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The FD analysis in Section 6.3 confirms, independently of how the schema was originally drawn, that the ECRAS relational design is already normalized to BCNF. Decomposing the natural flat application report by functional dependency alone converges back onto seven of the ten relations from the finalised ER diagram, and the remaining three (Rubric_Criteria, Bias_Audit_Logs, Propositions) need no further decomposition on their own. Alongside this, the hybrid retrieval design and the continuous bias-auditing layer show that ranking quality and regulatory compliance can be designed together rather than as separate concerns added on after the fact.

The design has since been carried through to a working implementation (Section 8). The schema was built in Oracle SQL and PL/SQL, with the business rules of Section 6.4 enforced by triggers, the ranking, interview-scoring and bias-audit logic held in database packages, and a Node.js and React application on top. The screens in Section 8.5 and the tests in Section 8.7 show the rules holding end to end: a mismatched job posting is refused, interview averages are derived from the transcript, and every bias-audit result is kept and cannot be deleted. The normalized schema supported these workflows with only a rank_position rename, a demographic_category column and an Audit_Trail table added.

9.2 Future Scope

- A follow-up-question agent for more adaptive interview probing.
- Multilingual, code-switching-aware transcription and rubric scoring.
- Graph-based skill-adjacency modelling for adjacent-but-not-identical skills.
- Moving demographic data into a separately permissioned schema, as described in Section 6.4.

9.3 References

- N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” EMNLP-IJCNLP, 2019.
- S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” Foundations and Trends in IR, 2009.
- O. Khattab and M. Zaharia, “ColBERT: Efficient and Effective Passage Search via Late Interaction over BERT,” SIGIR, 2020.
- G. V. Cormack et al., “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” SIGIR, 2009.
- New York City Local Law 144 of 2021, “Automated Employment Decision Tools,” N.Y.C. Admin. Code § 20-870 et seq.
- European Parliament and Council, “Regulation (EU) 2024/1689 (Artificial Intelligence Act),” Annex III, 2024.
- Equal Employment Opportunity Commission, “Uniform Guidelines on Employee Selection Procedures” (four-fifths rule), 29 C.F.R. § 1607.
- R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, Pearson Education.